

Darbas su studentais V3.0
2.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 include Directory Reference	9
5.2 src Directory Reference	9
6 Class Documentation	11
6.1 Studentas Class Reference	11
6.1.1 Constructor & Destructor Documentation	12
6.1.1.1 Studentas() [1/4]	12
6.1.1.2 Studentas() [2/4]	12
6.1.1.3 ~Studentas()	12
6.1.1.4 Studentas() [3/4]	12
6.1.1.5 Studentas() [4/4]	13
6.1.2 Member Function Documentation	13
6.1.2.1 addPazmys()	13
6.1.2.2 gautiVidurkiMediana()	13
6.1.2.3 gautiVidurkiVidutini()	13
6.1.2.4 getEgzaminoBalas()	13
6.1.2.5 getGalutinisMediana()	13
6.1.2.6 getGalutinisVidurkis()	13
6.1.2.7 getNamuDarbai()	13
6.1.2.8 getPavarde()	13
6.1.2.9 getPazymiai()	14
6.1.2.10 getVardas()	14
6.1.2.11 operator=() [1/2]	14
6.1.2.12 operator=() [2/2]	14
6.1.2.13 setEgzaminoBalas()	14
6.1.2.14 setGalutinisMediana()	14
6.1.2.15 setGalutinisVidurkis()	14
6.1.2.16 setNamuDarbai()	14
6.1.2.17 setPavarde()	14
6.1.2.18 setPazymiai()	15
6.1.2.19 setVardas()	15

6.1.2.20 surusiوتيPazymius()	15
6.1.2.21 Whoaml()	15
6.1.3 Friends And Related Symbol Documentation	15
6.1.3.1 operator<<	15
6.1.3.2 operator>>	15
6.2 Timer Class Reference	15
6.2.1 Detailed Description	16
6.2.2 Constructor & Destructor Documentation	16
6.2.2.1 Timer()	16
6.2.3 Member Function Documentation	16
6.2.3.1 elapsed()	16
6.2.3.2 reset()	16
6.3 Vector< T > Class Template Reference	16
6.3.1 Member Typedef Documentation	18
6.3.1.1 Allocator	18
6.3.1.2 const_iterator	19
6.3.1.3 const_reference	19
6.3.1.4 const_reverse_iterator	19
6.3.1.5 iterator	19
6.3.1.6 reference	19
6.3.1.7 reverse_iterator	19
6.3.1.8 size_type	19
6.3.1.9 type	19
6.3.2 Constructor & Destructor Documentation	20
6.3.2.1 Vector() [1/9]	20
6.3.2.2 Vector() [2/9]	20
6.3.2.3 Vector() [3/9]	20
6.3.2.4 Vector() [4/9]	20
6.3.2.5 Vector() [5/9]	20
6.3.2.6 Vector() [6/9]	20
6.3.2.7 Vector() [7/9]	20
6.3.2.8 Vector() [8/9]	21
6.3.2.9 Vector() [9/9]	21
6.3.2.10 ~Vector()	21
6.3.3 Member Function Documentation	21
6.3.3.1 append_range()	21
6.3.3.2 assign() [1/3]	21
6.3.3.3 assign() [2/3]	21
6.3.3.4 assign() [3/3]	21
6.3.3.5 assign_range()	22
6.3.3.6 at() [1/2]	22
6.3.3.7 at() [2/2]	22

6.3.3.8 back() [1/2]	22
6.3.3.9 back() [2/2]	22
6.3.3.10 begin() [1/2]	22
6.3.3.11 begin() [2/2]	22
6.3.3.12 capacity()	22
6.3.3.13 cbegin()	23
6.3.3.14 cend()	23
6.3.3.15 clear()	23
6.3.3.16 crbegin()	23
6.3.3.17 crend()	23
6.3.3.18 data_ptr() [1/2]	23
6.3.3.19 data_ptr() [2/2]	23
6.3.3.20 emplace()	23
6.3.3.21 emplace_back()	24
6.3.3.22 empty()	24
6.3.3.23 end() [1/2]	24
6.3.3.24 end() [2/2]	24
6.3.3.25 erase() [1/3]	24
6.3.3.26 erase() [2/3]	24
6.3.3.27 erase() [3/3]	24
6.3.3.28 front() [1/2]	24
6.3.3.29 front() [2/2]	25
6.3.3.30 get_allocator()	25
6.3.3.31 insert() [1/5]	25
6.3.3.32 insert() [2/5]	25
6.3.3.33 insert() [3/5]	25
6.3.3.34 insert() [4/5]	25
6.3.3.35 insert() [5/5]	25
6.3.3.36 insert_range()	26
6.3.3.37 max_size()	26
6.3.3.38 operator=() [1/3]	26
6.3.3.39 operator=() [2/3]	26
6.3.3.40 operator=() [3/3]	26
6.3.3.41 operator[]() [1/2]	26
6.3.3.42 operator[]() [2/2]	26
6.3.3.43 pop_back()	26
6.3.3.44 push_back() [1/2]	27
6.3.3.45 push_back() [2/2]	27
6.3.3.46 rbegin() [1/2]	27
6.3.3.47 rbegin() [2/2]	27
6.3.3.48 rend() [1/2]	27
6.3.3.49 rend() [2/2]	27

6.3.3.50 reserve()	27
6.3.3.51 resize() [1/2]	27
6.3.3.52 resize() [2/2]	28
6.3.3.53 shrink_to_fit()	28
6.3.3.54 size()	28
6.3.3.55 swap()	28
6.3.4 Friends And Related Symbol Documentation	28
6.3.4.1 operator!=	28
6.3.4.2 operator<	28
6.3.4.3 operator<=	28
6.3.4.4 operator==	29
6.3.4.5 operator>	29
6.3.4.6 operator>=	29
6.3.4.7 swap	29
6.4 Zmogus Class Reference	29
6.4.1 Detailed Description	30
6.4.2 Constructor & Destructor Documentation	30
6.4.2.1 Zmogus() [1/2]	30
6.4.2.2 Zmogus() [2/2]	30
6.4.2.3 ~Zmogus()	30
6.4.3 Member Function Documentation	30
6.4.3.1 getPavarde()	30
6.4.3.2 getVardas()	30
6.4.3.3 setPavarde()	31
6.4.3.4 setVardas()	31
6.4.3.5 Whoami()	31
6.4.4 Member Data Documentation	31
6.4.4.1 pavarde_	31
6.4.4.2 vardas_	31
7 File Documentation	33
7.1 include/functions.h File Reference	33
7.1.1 Detailed Description	34
7.1.2 Function Documentation	34
7.1.2.1 duomenuApdorojimoTestavimas()	34
7.1.2.2 failoDuomenuApdorojimas()	35
7.1.2.3 failoKurimoTestavimas()	35
7.1.2.4 failoNuskaitymas()	35
7.1.2.5 failuGeneravimas()	35
7.1.2.6 generuotiFaila()	35
7.1.2.7 generuotiPazymius()	35
7.1.2.8 generuotiStudenta()	35

7.1.2.9 isvestis()	35
7.1.2.10 isvestisFailas()	35
7.1.2.11 isvestisKonsole()	36
7.1.2.12 ivestisRanka()	36
7.1.2.13 nuskaitytiFaila()	36
7.1.2.14 nuskaitytiPavarde()	36
7.1.2.15 nuskaitytiPazymius()	36
7.1.2.16 nuskaitytiVarda()	36
7.1.2.17 ruleOfFiveTestas()	36
7.1.2.18 rusiuotiPagalMediana()	36
7.1.2.19 rusiuotiPagalPavarde()	36
7.1.2.20 rusiuotiPagalVarda()	37
7.1.2.21 rusiuotiPagalVidurki()	37
7.1.2.22 rusiuotiStudentus()	37
7.1.2.23 skaidytiStudentus1()	37
7.1.2.24 skaidytiStudentus2()	37
7.1.2.25 skaidytiStudentus3()	37
7.1.2.26 skaitytiSkaiciu()	37
7.1.2.27 skaitytiZodi()	38
7.1.2.28 studentuDuomenuApdorojimas()	38
7.1.2.29 surusiuotiPagalPasirinkima()	38
7.1.2.30 suskaiciuotiGalutini()	38
7.1.2.31 suskaiciuotiGalutinius()	38
7.1.2.32 suskaiciuotiStudentoGalutinius()	38
7.1.2.33 testuotiGreiti()	38
7.1.2.34 tikrinti()	38
7.2 functions.h	39
7.3 include/Studentas.h File Reference	39
7.3.1 Detailed Description	40
7.3.2 Typedef Documentation	40
7.3.2.1 Comparator	40
7.3.2.2 StudentuKonteineris	40
7.3.3 Function Documentation	40
7.3.3.1 surusiuotiStudentus() [1/2]	40
7.3.3.2 surusiuotiStudentus() [2/2]	41
7.4 Studentas.h	41
7.5 include/timer.h File Reference	43
7.6 timer.h	43
7.7 include/Vector.h File Reference	43
7.8 Vector.h	44
7.9 include/Zmogus.h File Reference	51
7.10 Zmogus.h	51

7.11 src/functions.cpp File Reference	52
7.11.1 Detailed Description	53
7.11.2 Function Documentation	53
7.11.2.1 duomenuApdorojimoTestavimas()	53
7.11.2.2 failoDuomenuApdorojimas()	53
7.11.2.3 failoKurimoTestavimas()	54
7.11.2.4 failoNuskaitymas()	54
7.11.2.5 failuGeneravimas()	54
7.11.2.6 generuotiFaila()	54
7.11.2.7 generuotiPazymius()	54
7.11.2.8 generuotiStudenta()	54
7.11.2.9 isvestis()	54
7.11.2.10 isvestisFailas()	54
7.11.2.11 isvestisKonsole()	54
7.11.2.12 ivestisRanka()	55
7.11.2.13 nuskaitytiFaila()	55
7.11.2.14 nuskaitytiPavarde()	55
7.11.2.15 nuskaitytiPazymius()	55
7.11.2.16 nuskaitytiVarda()	55
7.11.2.17 randomInt()	55
7.11.2.18 ruleOfFiveTestas()	55
7.11.2.19 rusiuotiPagalMediana()	55
7.11.2.20 rusiuotiPagalPavarde()	55
7.11.2.21 rusiuotiPagalVarda()	56
7.11.2.22 rusiuotiPagalVidurki()	56
7.11.2.23 rusiuotiStudentus()	56
7.11.2.24 skaidytiStudentus1()	56
7.11.2.25 skaidytiStudentus2()	56
7.11.2.26 skaidytiStudentus3()	56
7.11.2.27 skaitytiSkaiciu()	56
7.11.2.28 skaitytiZodi()	57
7.11.2.29 studentuDuomenuApdorojimas()	57
7.11.2.30 surusiuotiPagalPasirinkima()	57
7.11.2.31 suskaiciuotiGalutini()	57
7.11.2.32 suskaiciuotiGalutinius()	57
7.11.2.33 suskaiciuotiStudentoGalutinius()	57
7.11.2.34 testuotiGreiti()	57
7.11.2.35 tikrinti()	57
7.12 src/main.cpp File Reference	57
7.12.1 Function Documentation	58
7.12.1.1 main()	58
7.13 src/Studentas.cpp File Reference	58

7.13.1 Function Documentation	59
7.13.1.1 operator<<()	59
7.13.1.2 operator>>()	59
Index	61

Chapter 1

Directory Hierarchy

1.1 Directories

include	9
functions.h	33
Studentas.h	39
timer.h	43
Vector.h	43
Zmogus.h	51
src	9
functions.cpp	52
main.cpp	57
Studentas.cpp	58

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Timer	15
Vector< T >	16
Zmogus	29
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	11
Timer	
Timer klasė, skirta matuoti laiką tarp operacijų	15
Vector< T >	16
Zmogus	29

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/ functions.h	
Šiame faile yra deklaruojamos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą	33
include/ Studentas.h	
Šiame faile yra aprašoma Studentas klasė, kuri paveldi iš abstrakčios Zmogus klasės. Studentas klasė turi papildomų laukų, tokių kaip pažymiai, namų darbai, egzamino balas, galutinis vidurkis ir mediana. Taip pat yra aprašomi penki taisyklių (Rule of Five) metodai: konstruktoriumi, kopijavimo konstruktorius, kopijavimo priskyrimo operatorius, perkėlimo konstruktorius ir perkėlimo priskyrimo operatorius. Be to, yra aprašomi perdengti įvedimo ir išvedimo operatoriai	39
include/ timer.h	43
include/ Vector.h	43
include/ Zmogus.h	51
src/ functions.cpp	
Šiame faile yra įgyvendintos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą	52
src/ main.cpp	57
src/ Studentas.cpp	58

Chapter 5

Directory Documentation

5.1 include Directory Reference

Files

- file [functions.h](#)

Šiame faile yra deklaruojamos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

- file [Studentas.h](#)

Šiame faile yra aprašoma [Studentas](#) klasė, kuri paveldi iš abstrakčios [Zmogus](#) klasės. [Studentas](#) klasė turi papildomų laukų, tokių kaip pažymiai, namų darbai, egzamino balas, galutinis vidurkis ir mediana. Taip pat yra aprašomi penki taisyklių (Rule of Five) metodai: konstruktoriumi, kopijavimo konstruktorius, kopijavimo priskyrimo operatorius, perkėlimo konstruktorius ir perkėlimo priskyrimo operatorius. Be to, yra aprašomi perdengti įvedimo ir išvedimo operatoriai.

- file [timer.h](#)
- file [Vector.h](#)
- file [Zmogus.h](#)

5.2 src Directory Reference

Files

- file [functions.cpp](#)

Šiame faile yra įgyvendintos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

- file [main.cpp](#)
- file [Studentas.cpp](#)

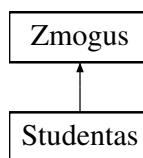
Chapter 6

Class Documentation

6.1 Studentas Class Reference

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (std::string vardas, std::string pavarde, int namuDarbai, std::vector< int > pazymiai, int egzaminoBalas)
- [~Studentas](#) ()
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other)
- void [WhoamI](#) () const override
- std::string [getVardas](#) () const
- std::string [getPavarde](#) () const
- int [getNamuDarbai](#) () const
- int [getEgzaminoBalas](#) () const
- double [getGalutinisVidurkis](#) () const
- double [getGalutinisMediana](#) () const
- const std::vector< int > & [getPazymiai](#) () const
- void [setVardas](#) (const std::string &vardas)
- void [setPavarde](#) (const std::string &pavarde)
- void [setEgzaminoBalas](#) (const int balas)
- void [setNamuDarbai](#) (const int nd)
- void [setGalutinisVidurkis](#) (const double vidurkis)
- void [setGalutinisMediana](#) (const double mediana)
- void [setPazymiai](#) (std::vector< int > pazymiai)
- void [addPazymys](#) (int pazymys)
- void [surusiuotiPazymius](#) ()
- double [gautiVidurkiVidutini](#) () const
- double [gautiVidurkiMediana](#) () const

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual [~Zmogus](#) ()
- std::string [getVardas](#) () const
- std::string [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Friends

- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)
- std::istream & [operator>>](#) (std::istream &in, [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas_](#)
- std::string [pavarde_](#)

6.1.1 Constructor & Destructor Documentation

6.1.1.1 [Studentas\(\)](#) [1/4]

```
Studentas::Studentas () [inline]
```

6.1.1.2 [Studentas\(\)](#) [2/4]

```
Studentas::Studentas (
    std::string vardas,
    std::string pavarde,
    int namuDarbai,
    std::vector< int > pazymiai,
    int egzaminoBalas) [inline]
```

6.1.1.3 [~Studentas\(\)](#)

```
Studentas::~~Studentas () [inline]
```

6.1.1.4 [Studentas\(\)](#) [3/4]

```
Studentas::Studentas (
    const Studentas & other) [inline]
```

6.1.1.5 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [inline]
```

6.1.2 Member Function Documentation

6.1.2.1 addPazymys()

```
void Studentas::addPazymys (  
    int pazymys) [inline]
```

6.1.2.2 gautiVidurkiMediana()

```
double Studentas::gautiVidurkiMediana () const
```

6.1.2.3 gautiVidurkiVidutini()

```
double Studentas::gautiVidurkiVidutini () const
```

6.1.2.4 getEgzaminoBalas()

```
int Studentas::getEgzaminoBalas () const [inline]
```

6.1.2.5 getGalutinisMediana()

```
double Studentas::getGalutinisMediana () const [inline]
```

6.1.2.6 getGalutinisVidurkis()

```
double Studentas::getGalutinisVidurkis () const [inline]
```

6.1.2.7 getNamuDarbai()

```
int Studentas::getNamuDarbai () const [inline]
```

6.1.2.8 getPavarde()

```
std::string Studentas::getPavarde () const [inline]
```

6.1.2.9 getPazymiai()

```
const std::vector< int > & Studentas::getPazymiai () const [inline]
```

6.1.2.10 getVardas()

```
std::string Studentas::getVardas () const [inline]
```

6.1.2.11 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other) [inline]
```

6.1.2.12 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [inline]
```

6.1.2.13 setEgzaminoBalas()

```
void Studentas::setEgzaminoBalas (
    const int balas) [inline]
```

6.1.2.14 setGalutinisMediana()

```
void Studentas::setGalutinisMediana (
    const double mediana) [inline]
```

6.1.2.15 setGalutinisVidurkis()

```
void Studentas::setGalutinisVidurkis (
    const double vidurkis) [inline]
```

6.1.2.16 setNamuDarbai()

```
void Studentas::setNamuDarbai (
    const int nd) [inline]
```

6.1.2.17 setPavarde()

```
void Studentas::setPavarde (
    const std::string & pavarde) [inline]
```

6.1.2.18 setPazymiai()

```
void Studentas::setPazymiai (
    std::vector< int > pazymiai) [inline]
```

6.1.2.19 setVardas()

```
void Studentas::setVardas (
    const std::string & vardas) [inline]
```

6.1.2.20 surusiuotiPazymius()

```
void Studentas::surusiuotiPazymius ()
```

6.1.2.21 WhoamI()

```
void Studentas::WhoamI () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

6.1.3 Friends And Related Symbol Documentation

6.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s) [friend]
```

6.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s) [friend]
```

The documentation for this class was generated from the following files:

- [include/Studentas.h](#)
- [src/Studentas.cpp](#)

6.2 Timer Class Reference

[Timer](#) klasė, skirta matuoti laiką tarp operacijų.

```
#include <timer.h>
```

Public Member Functions

- [Timer](#) ()
- void [reset](#) ()
- double [elapsed](#) () const

6.2.1 Detailed Description

[Timer](#) klasė, skirta matuoti laiką tarp operacijų.

Ši klasė naudoja C++11 chrono biblioteką, kad galėtų tiksliai matuoti laiką. Ji turi metodus laikui pradėti, iš naujo nustatyti ir gauti praėjusį laiką sekundėmis.

[Timer](#) klasė yra naudinga greičio testavimui ir optimizavimui, leidžiant lengvai matuoti, kiek laiko užtrunka tam tikros operacijos ar funkcijos vykdymas.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Timer()

```
Timer::Timer () [inline]
```

6.2.3 Member Function Documentation

6.2.3.1 elapsed()

```
double Timer::elapsed () const [inline]
```

6.2.3.2 reset()

```
void Timer::reset () [inline]
```

The documentation for this class was generated from the following file:

- include/[timer.h](#)

6.3 Vector< T > Class Template Reference

```
#include <Vector.h>
```

Public Types

- using `type` = T
- using `size_type` = std::size_t
- using `Allocator` = std::allocator<T>
- using `reference` = T &
- using `const_reference` = const T &
- using `iterator` = T *
- using `const_iterator` = const T *
- using `reverse_iterator` = std::reverse_iterator<iterator>
- using `const_reverse_iterator` = std::reverse_iterator<const_iterator>

Public Member Functions

- `Vector` ()
- `Vector` (size_type count)
- `Vector` (size_type count, const T &value)
- template<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
 `Vector` (InputIt first, InputIt last)
- `Vector` (const `Vector` &other, const `Allocator` &alloc)
- `Vector` (`Vector` &&other) noexcept
- `Vector` (`Vector` &&other, const `Allocator` &alloc)
- `Vector` (const `Vector` &other)
- `Vector` (std::initializer_list< T > init, const `Allocator` &alloc=`Allocator`())
- `~Vector` ()
- `Vector` & `operator=` (const `Vector` &other)
- `Vector` & `operator=` (`Vector` &&other) noexcept
- `Vector` & `operator=` (std::initializer_list< T > ilist)
- void `assign` (size_type count, const T &value)
- template<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
 void `assign` (InputIt first, InputIt last)
- void `assign` (std::initializer_list< T > ilist)
- template<class R>
 void `assign_range` (R &&rg)
- void `push_back` (const T &value)
- void `push_back` (T &&value)
- `reference operator[]` (size_type pos)
- `const_reference operator[]` (size_type pos) const
- size_type `size` () const noexcept
- size_type `capacity` () const noexcept
- void `reserve` (size_type new_cap)
- void `pop_back` ()
- void `clear` ()
- `iterator begin` ()
- `const_iterator begin` () const
- `const_iterator cbegin` () const noexcept
- `iterator end` ()
- `const_iterator end` () const
- `const_iterator cend` () const noexcept
- `reverse_iterator rbegin` ()
- `const_reverse_iterator rbegin` () const
- `const_reverse_iterator crbegin` () const noexcept
- `reverse_iterator rend` ()
- `const_reverse_iterator rend` () const

- [const_reverse_iterator](#) [crend](#) () const noexcept
- [reference](#) [front](#) ()
- [const_reference](#) [front](#) () const
- [reference](#) [back](#) ()
- [const_reference](#) [back](#) () const
- [template](#)<class... Args>
[iterator](#) [emplace](#) ([const_iterator](#) pos, Args &&...args)
- [template](#)<class... Args>
[reference](#) [emplace_back](#) (Args &&...args)
- [iterator](#) [insert](#) ([const_iterator](#) pos, const T &value)
- [iterator](#) [insert](#) ([const_iterator](#) pos, T &&value)
- [iterator](#) [insert](#) ([const_iterator](#) pos, [size_type](#) count, const T &value)
- [template](#)<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
[iterator](#) [insert](#) ([const_iterator](#) pos, InputIt first, InputIt last)
- [iterator](#) [insert](#) ([const_iterator](#) pos, std::initializer_list< T > ilist)
- [iterator](#) [erase](#) ([iterator](#) pos)
- [iterator](#) [erase](#) ([const_iterator](#) pos)
- [iterator](#) [erase](#) ([const_iterator](#) first, [const_iterator](#) last)
- void [shrink_to_fit](#) ()
- void [resize](#) ([size_type](#) count)
- void [resize](#) ([size_type](#) count, const T &value)
- bool [empty](#) () const noexcept
- T * [data_ptr](#) ()
- const T * [data_ptr](#) () const
- [reference](#) [at](#) ([size_type](#) pos)
- [const_reference](#) [at](#) ([size_type](#) pos) const
- void [swap](#) ([Vector](#) &other) noexcept(std::allocator_traits< [Allocator](#) >::propagate_on_container_swap::value||std::allocator_traits< [Allocator](#) >::is_always_equal::value)
- [size_type](#) [max_size](#) () const noexcept
- [template](#)<typename R>
[iterator](#) [insert_range](#) ([const_iterator](#) pos, R &&rg)
- [template](#)<typename R>
void [append_range](#) (R &&rg)
- [Allocator](#) [get_allocator](#) () const

Friends

- bool [operator==](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- bool [operator!=](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- bool [operator<](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- bool [operator<=](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- bool [operator>](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- bool [operator>=](#) (const [Vector](#) &lhs, const [Vector](#) &rhs)
- void [swap](#) ([Vector](#)< T > &a, [Vector](#)< T > &b) noexcept

6.3.1 Member Typedef Documentation

6.3.1.1 Allocator

```
template<typename T>
using Vector< T >::Allocator = std::allocator<T>
```

6.3.1.2 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T *
```

6.3.1.3 const_reference

```
template<typename T>
using Vector< T >::const_reference = const T &
```

6.3.1.4 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

6.3.1.5 iterator

```
template<typename T>
using Vector< T >::iterator = T *
```

6.3.1.6 reference

```
template<typename T>
using Vector< T >::reference = T &
```

6.3.1.7 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

6.3.1.8 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

6.3.1.9 type

```
template<typename T>
using Vector< T >::type = T
```

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Vector() [1/9]

```
template<typename T>
Vector< T >::Vector () [inline]
```

6.3.2.2 Vector() [2/9]

```
template<typename T>
Vector< T >::Vector (
    size_type count) [inline], [explicit]
```

6.3.2.3 Vector() [3/9]

```
template<typename T>
Vector< T >::Vector (
    size_type count,
    const T & value) [inline]
```

6.3.2.4 Vector() [4/9]

```
template<typename T>
template<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
Vector< T >::Vector (
    InputIt first,
    InputIt last) [inline]
```

6.3.2.5 Vector() [5/9]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other,
    const Allocator & alloc) [inline]
```

6.3.2.6 Vector() [6/9]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

6.3.2.7 Vector() [7/9]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other,
    const Allocator & alloc) [inline]
```

6.3.2.8 Vector() [8/9]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

6.3.2.9 Vector() [9/9]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > init,
    const Allocator & alloc = Allocator()) [inline]
```

6.3.2.10 ~Vector()

```
template<typename T>
Vector< T >::~Vector () [inline]
```

6.3.3 Member Function Documentation**6.3.3.1 append_range()**

```
template<typename T>
template<typename R>
void Vector< T >::append_range (
    R && rg) [inline]
```

6.3.3.2 assign() [1/3]

```
template<typename T>
template<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

6.3.3.3 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value) [inline]
```

6.3.3.4 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > ilist) [inline]
```

6.3.3.5 assign_range()

```
template<typename T>
template<class R>
void Vector< T >::assign_range (
    R && rg) [inline]
```

6.3.3.6 at() [1/2]

```
template<typename T>
reference Vector< T >::at (
    size_type pos) [inline]
```

6.3.3.7 at() [2/2]

```
template<typename T>
const_reference Vector< T >::at (
    size_type pos) const [inline]
```

6.3.3.8 back() [1/2]

```
template<typename T>
reference Vector< T >::back () [inline]
```

6.3.3.9 back() [2/2]

```
template<typename T>
const_reference Vector< T >::back () const [inline]
```

6.3.3.10 begin() [1/2]

```
template<typename T>
iterator Vector< T >::begin () [inline]
```

6.3.3.11 begin() [2/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline]
```

6.3.3.12 capacity()

```
template<typename T>
size_type Vector< T >::capacity () const [inline], [noexcept]
```

6.3.3.13 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline], [noexcept]
```

6.3.3.14 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline], [noexcept]
```

6.3.3.15 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

6.3.3.16 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline], [noexcept]
```

6.3.3.17 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline], [noexcept]
```

6.3.3.18 data_ptr() [1/2]

```
template<typename T>
T * Vector< T >::data_ptr () [inline]
```

6.3.3.19 data_ptr() [2/2]

```
template<typename T>
const T * Vector< T >::data_ptr () const [inline]
```

6.3.3.20 emplace()

```
template<typename T>
template<class... Args>
iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

6.3.3.21 `emplace_back()`

```
template<typename T>
template<class... Args>
reference Vector< T >::emplace_back (
    Args &&... args) [inline]
```

6.3.3.22 `empty()`

```
template<typename T>
bool Vector< T >::empty () const [inline], [noexcept]
```

6.3.3.23 `end()` [1/2]

```
template<typename T>
iterator Vector< T >::end () [inline]
```

6.3.3.24 `end()` [2/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline]
```

6.3.3.25 `erase()` [1/3]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

6.3.3.26 `erase()` [2/3]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

6.3.3.27 `erase()` [3/3]

```
template<typename T>
iterator Vector< T >::erase (
    iterator pos) [inline]
```

6.3.3.28 `front()` [1/2]

```
template<typename T>
reference Vector< T >::front () [inline]
```

6.3.3.29 front() [2/2]

```
template<typename T>
const_reference Vector< T >::front () const [inline]
```

6.3.3.30 get_allocator()

```
template<typename T>
Allocator Vector< T >::get_allocator () const [inline]
```

6.3.3.31 insert() [1/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value) [inline]
```

6.3.3.32 insert() [2/5]

```
template<typename T>
template<class InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last) [inline]
```

6.3.3.33 insert() [3/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value) [inline]
```

6.3.3.34 insert() [4/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    std::initializer_list< T > ilist) [inline]
```

6.3.3.35 insert() [5/5]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    T && value) [inline]
```

6.3.3.36 insert_range()

```
template<typename T>
template<typename R>
iterator Vector< T >::insert_range (
    const_iterator pos,
    R && rg) [inline]
```

6.3.3.37 max_size()

```
template<typename T>
size_type Vector< T >::max_size () const [inline], [noexcept]
```

6.3.3.38 operator=() [1/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

6.3.3.39 operator=() [2/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    std::initializer_list< T > ilist) [inline]
```

6.3.3.40 operator=() [3/3]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

6.3.3.41 operator[]() [1/2]

```
template<typename T>
reference Vector< T >::operator[] (
    size_type pos) [inline]
```

6.3.3.42 operator[]() [2/2]

```
template<typename T>
const_reference Vector< T >::operator[] (
    size_type pos) const [inline]
```

6.3.3.43 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

6.3.3.44 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

6.3.3.45 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

6.3.3.46 rbegin() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline]
```

6.3.3.47 rbegin() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline]
```

6.3.3.48 rend() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline]
```

6.3.3.49 rend() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline]
```

6.3.3.50 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap) [inline]
```

6.3.3.51 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count) [inline]
```

6.3.3.52 `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const T & value) [inline]
```

6.3.3.53 `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

6.3.3.54 `size()`

```
template<typename T>
size_type Vector< T >::size () const [inline], [noexcept]
```

6.3.3.55 `swap()`

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

6.3.4 Friends And Related Symbol Documentation**6.3.4.1** `operator"!="`

```
template<typename T>
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.2 `operator<`

```
template<typename T>
bool operator< (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.3 `operator<=`

```
template<typename T>
bool operator<= (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.4 operator==

```
template<typename T>
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.5 operator>

```
template<typename T>
bool operator> (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.6 operator>=

```
template<typename T>
bool operator>= (
    const Vector< T > & lhs,
    const Vector< T > & rhs) [friend]
```

6.3.4.7 swap

```
template<typename T>
void swap (
    Vector< T > & a,
    Vector< T > & b) [friend]
```

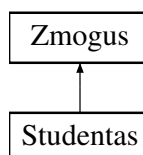
The documentation for this class was generated from the following file:

- include/[Vector.h](#)

6.4 Zmogus Class Reference

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual [~Zmogus](#) ()
- virtual void [Whoaml](#) () const =0
- std::string [getVardas](#) () const
- std::string [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Protected Attributes

- std::string [vardas_](#)
- std::string [pavarde_](#)

6.4.1 Detailed Description

Abstrakti bazinė klasė, atstovaujanti žmogų.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

6.4.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const std::string & vardas,  
    const std::string & pavarde) [inline]
```

6.4.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [inline], [virtual]
```

6.4.3 Member Function Documentation

6.4.3.1 getPavarde()

```
std::string Zmogus::getPavarde () const [inline]
```

6.4.3.2 getVardas()

```
std::string Zmogus::getVardas () const [inline]
```

6.4.3.3 setPavarde()

```
void Zmogus::setPavarde (  
    const std::string & p) [inline]
```

6.4.3.4 setVardas()

```
void Zmogus::setVardas (  
    const std::string & v) [inline]
```

6.4.3.5 WhoamI()

```
virtual void Zmogus::WhoamI () const [pure virtual]
```

Implemented in [Studentas](#).

6.4.4 Member Data Documentation

6.4.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

6.4.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

The documentation for this class was generated from the following file:

- include/[Zmogus.h](#)

Chapter 7

File Documentation

7.1 include/functions.h File Reference

Šiame faile yra deklaruojamos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

```
#include "Studentas.h"
#include <algorithm>
#include <cmath>
#include <deque>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
```

Functions

- [Studentas ivestisRanka](#) ()
- [Studentas generuotiPazymius](#) ()
- [Studentas generuotiStudenta](#) ()
- void [isvestis](#) (const [StudentuKonteineris](#) &studentai, bool arMediana)
- void [isvestisKonsole](#) (const [StudentuKonteineris](#) &studentai)
- void [isvestisFailas](#) (const [StudentuKonteineris](#) &studentai, std::string failoPavadinimas)
- std::string [skaitytiZodi](#) (const std::string &pranesimas)
- std::string [nuskaitytiVarda](#) ()
- std::string [nuskaitytiPavarde](#) ()
- bool [suskaiciuotiGalutini](#) ([StudentuKonteineris](#) &studentai)
- void [suskaiciuotiStudentoGalutinius](#) ([Studentas](#) &studentas)
- void [suskaiciuotiGalutinius](#) ([StudentuKonteineris](#) &studentai)
- std::vector< int > [nuskaitytiPazymius](#) (std::istream &is, int namuDarbai)
- void [nuskaitytiFaila](#) ([StudentuKonteineris](#) &studentai, const std::string &failoPavadinimas)

- bool `rusiuotiPagalVarda` (const `Studentas` &a, const `Studentas` &b)
- bool `rusiuotiPagalPavarde` (const `Studentas` &a, const `Studentas` &b)
- bool `rusiuotiPagalVidurki` (const `Studentas` &a, const `Studentas` &b)
- bool `rusiuotiPagalMediana` (const `Studentas` &a, const `Studentas` &b)
- void `rusiuotiStudentus` (`StudentuKonteineris` &studentai, int input)
- void `surusiuotiPagalPasirinkima` (`StudentuKonteineris` &studentai)
- int `skaitytiSkaiciu` (const std::string &pranesimas, int min, int max)
- std::string `generuotiFaila` (int studentuSkaicius, int namuDarbuSkaicius)
- void `skaitytiStudentus1` (`StudentuKonteineris` &studentai, `StudentuKonteineris` &vargsiukai, `StudentuKonteineris` &kietiakai)
- void `skaitytiStudentus2` (`StudentuKonteineris` &studentai, `StudentuKonteineris` &kietiakai)
- void `skaitytiStudentus3` (`StudentuKonteineris` &studentai, `StudentuKonteineris` &vargsiukai, `StudentuKonteineris` &kietiakai)
- void `failuGeneravimas` ()
- void `failoKurimoTestavimas` ()
- void `duomenuApdorojimoTestavimas` ()
- void `tikrinti` (bool condition)
- void `ruleOfFiveTestas` ()
- void `testuotiGreiti` ()
- void `failoNuskaitymas` (`StudentuKonteineris` &studentai)
- void `failoDuomenuApdorojimas` (`StudentuKonteineris` &studentai)
- void `studentuDuomenuApdorojimas` (`StudentuKonteineris` &studentai)

7.1.1 Detailed Description

Šiame faile yra deklaruojamos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

Šios funkcijos apima:

- Duomenų įvedimą ranka,
- Atsitiktinių pažymių generavimą,
- Studentų vardų, pavardžių ir pažymių generavimą,
- Failų nuskaitymą ir duomenų apdorojimą,
- Duomenų išvedimą į konsolę ir failus,
- Studentų rūšiavimą pagal įvairius kriterijus,
- Greičio testavimą naudojant `Timer` klasę.

Šios funkcijos yra pagrindinės programos dalys, leidžiančios vartotojui efektyviai dirbti su studentų duomenimis ir atlikti įvairias operacijas su jais.

7.1.2 Function Documentation

7.1.2.1 `duomenuApdorojimoTestavimas()`

```
void duomenuApdorojimoTestavimas ()
```

7.1.2.2 failoDuomenuApdorojimas()

```
void failoDuomenuApdorojimas (  
    StudentuKonteineris & studentai)
```

7.1.2.3 failoKurimoTestavimas()

```
void failoKurimoTestavimas ()
```

7.1.2.4 failoNuskaitymas()

```
void failoNuskaitymas (  
    StudentuKonteineris & studentai)
```

7.1.2.5 failuGeneravimas()

```
void failuGeneravimas ()
```

7.1.2.6 generuotiFaila()

```
std::string generuotiFaila (  
    int studentuSkaicius,  
    int namuDarbuSkaicius)
```

7.1.2.7 generuotiPazymius()

```
Studentas generuotiPazymius ()
```

7.1.2.8 generuotiStudenta()

```
Studentas generuotiStudenta ()
```

7.1.2.9 isvestis()

```
void isvestis (  
    const StudentuKonteineris & studentai,  
    bool arMediana)
```

7.1.2.10 isvestisFailas()

```
void isvestisFailas (  
    const StudentuKonteineris & studentai,  
    std::string failoPavadinimas)
```

7.1.2.11 isvestisKonsole()

```
void isvestisKonsole (
    const StudentuKonteineris & studentai)
```

7.1.2.12 igestisRanka()

```
Studentas igestisRanka ()
```

7.1.2.13 nuskaitytiFaila()

```
void nuskaitytiFaila (
    StudentuKonteineris & studentai,
    const std::string & failoPavadinimas)
```

7.1.2.14 nuskaitytiPavarde()

```
std::string nuskaitytiPavarde ()
```

7.1.2.15 nuskaitytiPazymius()

```
std::vector< int > nuskaitytiPazymius (
    std::istream & is,
    int namuDarbai)
```

7.1.2.16 nuskaitytiVarda()

```
std::string nuskaitytiVarda ()
```

7.1.2.17 ruleOfFiveTestas()

```
void ruleOfFiveTestas ()
```

7.1.2.18 rusiuotiPagalMediana()

```
bool rusiuotiPagalMediana (
    const Studentas & a,
    const Studentas & b)
```

7.1.2.19 rusiuotiPagalPavarde()

```
bool rusiuotiPagalPavarde (
    const Studentas & a,
    const Studentas & b)
```

7.1.2.20 rusiuotiPagalVarda()

```
bool rusiuotiPagalVarda (  
    const Studentas & a,  
    const Studentas & b)
```

7.1.2.21 rusiuotiPagalVidurki()

```
bool rusiuotiPagalVidurki (  
    const Studentas & a,  
    const Studentas & b)
```

7.1.2.22 rusiuotiStudentus()

```
void rusiuotiStudentus (  
    StudentuKonteineris & studentai,  
    int input)
```

7.1.2.23 skaidytiStudentus1()

```
void skaidytiStudentus1 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & vargsiukai,  
    StudentuKonteineris & kietiakai)
```

7.1.2.24 skaidytiStudentus2()

```
void skaidytiStudentus2 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & kietiakai)
```

7.1.2.25 skaidytiStudentus3()

```
void skaidytiStudentus3 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & vargsiukai,  
    StudentuKonteineris & kietiakai)
```

7.1.2.26 skaitytiSkaiciu()

```
int skaitytiSkaiciu (  
    const std::string & pranesimas,  
    int min,  
    int max)
```

7.1.2.27 skaitytiZodi()

```
std::string skaitytiZodi (  
    const std::string & pranesimas)
```

7.1.2.28 studentuDuomenuApdorojimas()

```
void studentuDuomenuApdorojimas (  
    StudentuKonteineris & studentai)
```

7.1.2.29 surusiuotiPagalPasirinkima()

```
void surusiuotiPagalPasirinkima (  
    StudentuKonteineris & studentai)
```

7.1.2.30 suskaiciuotiGalutini()

```
bool suskaiciuotiGalutini (  
    StudentuKonteineris & studentai)
```

7.1.2.31 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (  
    StudentuKonteineris & studentai)
```

7.1.2.32 suskaiciuotiStudentoGalutinius()

```
void suskaiciuotiStudentoGalutinius (  
    Studentas & studentas)
```

7.1.2.33 testuotiGreiti()

```
void testuotiGreiti ()
```

7.1.2.34 tikrinti()

```
void tikrinti (  
    bool condition)
```

7.2 functions.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "Studentas.h"
00003 #include <algorithm>
00004 #include <cmath>
00005 #include <deque>
00006 #include <filesystem>
00007 #include <fstream>
00008 #include <iomanip>
00009 #include <iostream>
00010 #include <list>
00011 #include <random>
00012 #include <sstream>
00013 #include <stdexcept>
00014 #include <string>
00015 #include <vector>
00016
00037
00038 Studentas ivestisRanka();
00039 Studentas generuotiPazymius();
00040 Studentas generuotiStudenta();
00041 void isvestis(const StudentuKonteineris &studentai, bool arMediana);
00042 void isvestisKonsole(const StudentuKonteineris &studentai);
00043 void isvestisFailas(const StudentuKonteineris &studentai,
00044                    std::string failoPavadinimas);
00045 std::string skaitytiZodi(const std::string &pranesimas);
00046 std::string nuskaitytiVarda();
00047 std::string nuskaitytiPavarde();
00048 bool suskaiciuotiGalutini(StudentuKonteineris &studentai);
00049 void suskaiciuotiStudentoGalutinius(Studentas &studentas);
00050 void suskaiciuotiGalutinius(StudentuKonteineris &studentai);
00051 std::vector<int> nuskaitytiPazymius(std::istream &is, int namuDarbai);
00052 void nuskaitytiFaila(StudentuKonteineris &studentai,
00053                      const std::string &failoPavadinimas);
00054 bool rusiuotiPagalVarda(const Studentas &a, const Studentas &b);
00055 bool rusiuotiPagalPavarde(const Studentas &a, const Studentas &b);
00056 bool rusiuotiPagalVidurki(const Studentas &a, const Studentas &b);
00057 bool rusiuotiPagalMediana(const Studentas &a, const Studentas &b);
00058 void rusiuotiStudentus(StudentuKonteineris &studentai, int input);
00059 void surusiuotiPagalPasirinkima(StudentuKonteineris &studentai);
00060 int skaitytiSkaiciu(const std::string &pranesimas, int min, int max);
00061 std::string generuotiFaila(int studentuSkaicius, int namuDarbuSkaicius);
00062 void skaidytiStudentus1(StudentuKonteineris &studentai,
00063                          StudentuKonteineris &vargsiukai,
00064                          StudentuKonteineris &kietiakai);
00065 void skaidytiStudentus2(StudentuKonteineris &studentai,
00066                          StudentuKonteineris &kietiakai);
00067 void skaidytiStudentus3(StudentuKonteineris &studentai,
00068                          StudentuKonteineris &vargsiukai,
00069                          StudentuKonteineris &kietiakai);
00070 void failuGeneravimas();
00071 void failoKurimoTestavimas();
00072 void duomenuApdorojimoTestavimas();
00073 void tikrinti(bool condition);
00074 void ruleOfFiveTestas();
00075 void testuotiGreiti();
00076 void failoNuskaitymas(StudentuKonteineris &studentai);
00077 void failoDuomenuApdorojimas(StudentuKonteineris &studentai);
00078 void studentuDuomenuApdorojimas(StudentuKonteineris &studentai);
```

7.3 include/Studentas.h File Reference

Šiame faile yra aprašoma [Studentas](#) klasė, kuri paveldi iš abstrakčios [Zmogus](#) klasės. [Studentas](#) klasė turi papildomų laukų, tokių kaip pažymiai, namų darbai, egzamino balas, galutinis vidurkis ir mediana. Taip pat yra aprašomi penki taisyklių (Rule of Five) metodai: konstruktoriumi, kopijavimo konstruktorius, kopijavimo priskyrimo operatorius, perkėlimo konstruktorius ir perkėlimo priskyrimo operatorius. Be to, yra aprašomi perdengti įvedimo ir išvedimo operatoriai.

```
#include "Zmogus.h"
#include <iostream>
#include <list>
#include <string>
#include <vector>
```

Classes

- class [Studentas](#)

Typedefs

- using [StudentuKonteineris](#) = std::vector<[Studentas](#)>
- using [Comparator](#) = bool (*)(const [Studentas](#) &, const [Studentas](#) &)

Functions

- template<typename Konteineris>
void [surusiuotiStudentus](#) (Konteineris &studentai, [Comparator](#) comparator)
- template<> void [surusiuotiStudentus](#) (std::list< [Studentas](#) > &studentai, [Comparator](#) comparator)

7.3.1 Detailed Description

Šiame faile yra aprašoma [Studentas](#) klasė, kuri paveldi iš abstrakčios [Zmogus](#) klasės. [Studentas](#) klasė turi papildomų laukų, tokių kaip pažymiai, namų darbai, egzamino balas, galutinis vidurkis ir mediana. Taip pat yra aprašomi penki taisyklių (Rule of Five) metodai: konstruktoriumi, kopijavimo konstruktorius, kopijavimo priskyrimo operatorius, perkėlimo konstruktorius ir perkėlimo priskyrimo operatorius. Be to, yra aprašomi perdengti įvedimo ir išvedimo operatoriai.

[Studentas](#) klasė taip pat turi metodus galutiniam vidurkiui ir medianai apskaičiuoti, bei metodą [surusiuotiPazymius](#), kuris rūšiuoja pažymius.

Ši klasė yra skirta naudoti su įvairiais konteineriais, tokiais kaip vector, list ar deque, ir gali būti lengvai išplėsta su papildomais metodais ar laukais pagal poreikį.

7.3.2 Typedef Documentation

7.3.2.1 Comparator

```
using Comparator = bool (*)(const Studentas &, const Studentas &)
```

7.3.2.2 StudentuKonteineris

```
using StudentuKonteineris = std::vector<Studentas>
```

7.3.3 Function Documentation

7.3.3.1 surusiuotiStudentus() [1/2]

```
template<typename Konteineris>  
void surusiuotiStudentus (  
    Konteineris & studentai,  
    Comparator comparator) [inline]
```

7.3.3.2 surusiuotiStudentus() [2/2]

```
template<>
void surusiuotiStudentus (
    std::list< Studentas > & studentai,
    Comparator comparator) [inline]
```

7.4 Studentas.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "Zmogus.h"
00003 #include <iostream>
00004 #include <list>
00005 #include <string>
00006 #include <vector>
00007
00026 class Studentas : public Zmogus {
00027 private:
00028     std::vector<int> pazymiai_ = {};
00029     int namuDarbai_ = 0;
00030     int egzaminoBalas_ = 0;
00031     double galutinisVidurkis_ = 0.0;
00032     double galutinisMediana_ = 0.0;
00033
00034 public:
00035     Studentas()
00036         : Zmogus(), namuDarbai_(0), egzaminoBalas_(0), galutinisMediana_(0.0),
00037           galutinisVidurkis_(0.0) {}
00038
00039     Studentas(std::string vardas, std::string pavarde, int namuDarbai,
00040               std::vector<int> pazymiai, int egzaminoBalas)
00041         : Zmogus(vardas, pavarde), namuDarbai_(namuDarbai),
00042           pazymiai_(std::move(pazymiai)), egzaminoBalas_(egzaminoBalas) {}
00043
00044     ~Studentas() {
00045         pazymiai_.clear();
00046         namuDarbai_ = 0;
00047         egzaminoBalas_ = 0;
00048         galutinisVidurkis_ = 0.0;
00049         galutinisMediana_ = 0.0;
00050     }
00051
00052     // Kopijavimo konstruktorius
00053     Studentas(const Studentas &other)
00054         : Zmogus(other.vardas_, other.pavarde_), namuDarbai_(other.namuDarbai_),
00055           pazymiai_(other.pazymiai_), egzaminoBalas_(other.egzaminoBalas_),
00056           galutinisVidurkis_(other.galutinisVidurkis_),
00057           galutinisMediana_(other.galutinisMediana_) {}
00058
00059     // Kopijavimo priskyrimo operatorius
00060     Studentas &operator=(const Studentas &other) {
00061         if (this == &other) {
00062             return *this;
00063         }
00064
00065         vardas_ = other.vardas_;
00066         pavarde_ = other.pavarde_;
00067         namuDarbai_ = other.namuDarbai_;
00068         pazymiai_ = other.pazymiai_;
00069         egzaminoBalas_ = other.egzaminoBalas_;
00070         galutinisVidurkis_ = other.galutinisVidurkis_;
00071         galutinisMediana_ = other.galutinisMediana_;
00072
00073         return *this;
00074     }
00075
00076     // Perkelimo konstruktorius
00077     Studentas(Studentas &&other)
00078         : Zmogus(std::move(other.vardas_), std::move(other.pavarde_)),
00079           namuDarbai_(other.namuDarbai_), pazymiai_(std::move(other.pazymiai_)),
00080           egzaminoBalas_(other.egzaminoBalas_),
00081           galutinisVidurkis_(other.galutinisVidurkis_),
00082           galutinisMediana_(other.galutinisMediana_) {
00083
00084         other.vardas_.clear();
00085         other.pavarde_.clear();
```

```

00086     other.namuDarbai_ = 0;
00087     other.egzaminoBalas_ = 0;
00088     other.galutinisVidurkis_ = 0.0;
00089     other.galutinisMediana_ = 0.0;
00090 }
00091
00092 // Perkelimo priskyrimo operatorius
00093 Studentas &operator=(Studentas &&other) {
00094     if (this == &other) {
00095         return *this;
00096     }
00097
00098     vardas_ = std::move(other.vardas_);
00099     pavarde_ = std::move(other.pavarde_);
00100     namuDarbai_ = other.namuDarbai_;
00101     pazymiai_ = std::move(other.pazymiai_);
00102     egzaminoBalas_ = other.egzaminoBalas_;
00103     galutinisVidurkis_ = other.galutinisVidurkis_;
00104     galutinisMediana_ = other.galutinisMediana_;
00105
00106     other.vardas_.clear();
00107     other.pavarde_.clear();
00108     other.namuDarbai_ = 0;
00109     other.egzaminoBalas_ = 0;
00110     other.galutinisVidurkis_ = 0.0;
00111     other.galutinisMediana_ = 0.0;
00112
00113     return *this;
00114 }
00115
00116 void WhoamI() const override {
00117     std::cout << vardas_ << " " << pavarde_ << std::endl;
00118 }
00119
00120 // Perdengtas išvedimo operatorius
00121 friend std::ostream &operator<<(std::ostream &out, const Studentas &s);
00122 // Perdengtas įvedimo operatorius
00123 friend std::istream &operator>>(std::istream &in, Studentas &s);
00124
00125 std::string getVardas() const { return vardas_; }
00126 std::string getPavarde() const { return pavarde_; }
00127 int getNamuDarbai() const { return namuDarbai_; }
00128 int getEgzaminoBalas() const { return egzaminoBalas_; }
00129 double getGalutinisVidurkis() const { return galutinisVidurkis_; }
00130 double getGalutinisMediana() const { return galutinisMediana_; }
00131 const std::vector<int> &getPazymiai() const { return pazymiai_; }
00132
00133 void setVardas(const std::string &vardas) { vardas_ = vardas; }
00134 void setPavarde(const std::string &pavarde) { pavarde_ = pavarde; }
00135 void setEgzaminoBalas(const int balas) { egzaminoBalas_ = balas; }
00136 void setNamuDarbai(const int nd) { namuDarbai_ = nd; }
00137 void setGalutinisVidurkis(const double vidurkis) {
00138     galutinisVidurkis_ = vidurkis;
00139 }
00140 void setGalutinisMediana(const double mediana) {
00141     galutinisMediana_ = mediana;
00142 }
00143 void setPazymiai(std::vector<int> pazymiai) {
00144     pazymiai_ = std::move(pazymiai);
00145 }
00146 void addPazymys(int pazymys) { pazymiai_.push_back(pazymys); }
00147
00148 void surusiuotiPazymius();
00149 double gautiVidurkiVidutini() const;
00150 double gautiVidurkiMediana() const;
00151 };
00152
00153 using StudentuKonteineris = std::vector<Studentas>;
00154 // using StudentuKonteineris = std::list<Studentas>;
00155 // using StudentuKonteineris = std::deque<Studentas>;
00156
00157 // Rodykle i bet kokia funkcija, kuri priima du Studentas objektus ir grazina
00158 // bool reiksme
00159 using Comparator = bool (*)(const Studentas &, const Studentas &);
00160
00161 template <typename Konteineris>
00162 inline void surusiuotiStudentus(Konteineris &studentai, Comparator comparator) {
00163     sort(studentai.begin(), studentai.end(), comparator);
00164 }
00165
00166 template <>
00167 inline void surusiuotiStudentus(std::list<Studentas> &studentai,
00168     Comparator comparator) {
00169     studentai.sort(comparator);
00170 }

```

7.5 include/timer.h File Reference

```
#include <chrono>
```

Classes

- class [Timer](#)

Timer klasė, skirta matuoti laiką tarp operacijų.

7.6 timer.h

[Go to the documentation of this file.](#)

```
00001 #include <chrono>
00002 using std::chrono::duration;
00003 using std::chrono::high_resolution_clock;
00004 using std::chrono::time_point;
00005
00018 class Timer {
00019     using hrClock = high_resolution_clock;
00020
00021 private:
00022     time_point<hrClock> start;
00023
00024 public:
00025     Timer() : start{hrClock::now()} {}
00026     void reset() { start = hrClock::now(); }
00027     double elapsed() const {
00028         return duration<double>(hrClock::now() - start).count();
00029     }
00030 };
```

7.7 include/Vector.h File Reference

```
#include <algorithm>
#include <cstdint>
#include <initializer_list>
#include <iterator>
#include <memory>
#include <stdexcept>
#include <type_traits>
#include <utility>
```

Classes

- class [Vector< T >](#)

7.8 Vector.h

[Go to the documentation of this file.](#)

```

00001 #ifndef Vector_H
00002 #define Vector_H
00003
00004 #include <algorithm>
00005 #include <cstddef>
00006 #include <initializer_list>
00007 #include <iterator>
00008 #include <memory>
00009 #include <stdexcept>
00010 #include <type_traits>
00011 #include <utility>
00012
00013 template <typename T> class Vector {
00014 private:
00015     T *data;
00016     size_t Asize;
00017     size_t Acapacity;
00018     std::allocator<T> alloc;
00019
00020 public:
00021     using type = T;
00022     using size_type = std::size_t;
00023     using Allocator = std::allocator<T>;
00024
00025     using reference = T &;
00026     using const_reference = const T &;
00027     using iterator = T *;
00028     using const_iterator = const T *;
00029     using reverse_iterator = std::reverse_iterator<iterator>;
00030     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00031
00032     Vector() : data(nullptr), Asize(0), Acapacity(0) {}
00033
00034     explicit Vector(size_type count) : Asize(count), Acapacity(count) {
00035         data = alloc.allocate(count);
00036         for (size_type i = 0; i < count; ++i) {
00037             std::allocator_traits<Allocator>::construct(alloc, data + i);
00038         }
00039     }
00040
00041     Vector(size_type count, const T &value) : Asize(count), Acapacity(count) {
00042         data = alloc.allocate(count);
00043         for (size_type i = 0; i < count; ++i) {
00044             std::allocator_traits<Allocator>::construct(alloc, data + i, value);
00045         }
00046     }
00047     template <class InputIt, typename = typename std::enable_if<
00048         !std::is_integral<InputIt>::value>::type>
00049     Vector(InputIt first, InputIt last) {
00050         Asize = std::distance(first, last);
00051         Acapacity = Asize;
00052
00053         data = alloc.allocate(Acapacity);
00054
00055         size_type i = 0;
00056
00057         try {
00058             for (; first != last; ++first, ++i) {
00059                 std::allocator_traits<Allocator>::construct(alloc, data + i, *first);
00060             }
00061         } catch (...) {
00062             for (size_type j = 0; j < i; ++j) {
00063                 std::allocator_traits<Allocator>::destroy(alloc, data + j);
00064             }
00065
00066             alloc.deallocate(data, Acapacity);
00067             throw;
00068         }
00069     }
00070
00071     Vector(const Vector &other, const Allocator &alloc)
00072         : Asize(other.Asize), Acapacity(other.Acapacity), alloc(alloc) {
00073         data = this->alloc.allocate(Acapacity);
00074
00075         for (size_type i = 0; i < Asize; ++i)
00076             std::allocator_traits<Allocator>::construct(this->alloc, data + i,
00077                                                         other.data[i]);
00078     }
00079
00080     Vector(Vector &&other) noexcept
00081         : data(other.data), Asize(other.Asize), Acapacity(other.Acapacity),
00082         alloc(std::move(other.alloc)) {

```

```

00083     other.data = nullptr;
00084     other.Asize = 0;
00085     other.Acapacity = 0;
00086 }
00087
00088 Vector(Vector &&other, const Allocator &alloc) {
00089     data = other.data;
00090     Asize = other.Asize;
00091     Acapacity = other.Acapacity;
00092     this->alloc = alloc;
00093     other.data = nullptr;
00094     other.Asize = 0;
00095     other.Acapacity = 0;
00096 }
00097
00098 Vector(const Vector &other) : Asize(other.Asize), Acapacity(other.Acapacity) {
00099     data = alloc.allocate(Acapacity);
00100
00101     for (size_type i = 0; i < Asize; ++i) {
00102         std::allocator_traits<Allocator>::construct(alloc, data + i,
00103                                                     other.data[i]);
00104     }
00105 }
00106
00107 Vector(std::initializer_list<T> init, const Allocator &alloc = Allocator())
00108     : Asize(init.size()), Acapacity(init.size()), alloc(alloc) {
00109     data = this->alloc.allocate(Acapacity);
00110
00111     size_type i = 0;
00112     for (const auto &v : init)
00113         std::allocator_traits<Allocator>::construct(this->alloc, data + i++, v);
00114 }
00115
00116 ~Vector() {
00117     clear();
00118     if (data)
00119         alloc.deallocate(data, Acapacity);
00120 }
00121
00122 Vector &operator=(const Vector &other) {
00123     if (this == &other)
00124         return *this;
00125
00126     T *newData = alloc.allocate(other.Acapacity);
00127     size_type i = 0;
00128
00129     try {
00130         for (; i < other.Asize; ++i) {
00131             std::allocator_traits<Allocator>::construct(alloc, newData + i,
00132                                                         other.data[i]);
00133         }
00134     } catch (...) {
00135         for (size_type j = 0; j < i; ++j)
00136             std::allocator_traits<Allocator>::destroy(alloc, newData + j);
00137
00138         alloc.deallocate(newData, other.Acapacity);
00139         throw;
00140     }
00141     clear();
00142     if (data)
00143         alloc.deallocate(data, Acapacity);
00144
00145     data = newData;
00146     Asize = other.Asize;
00147     Acapacity = other.Acapacity;
00148
00149     return *this;
00150 }
00151
00152 Vector &operator=(Vector &&other) noexcept {
00153     if (this != &other) {
00154         clear();
00155         if (data)
00156             alloc.deallocate(data, Acapacity);
00157
00158         data = other.data;
00159         Asize = other.Asize;
00160         Acapacity = other.Acapacity;
00161         alloc = std::move(other.alloc);
00162
00163         other.data = nullptr;
00164         other.Asize = 0;
00165         other.Acapacity = 0;
00166     }
00167     return *this;
00168 }
00169

```

```

00170 Vector &operator=(std::initializer_list<T> ilist) {
00171
00172     clear();
00173     if (data)
00174         alloc.deallocate(data, Acapacity);
00175
00176     Asize = ilist.size();
00177     Acapacity = ilist.size();
00178     data = alloc.allocate(Acapacity);
00179
00180     size_t i = 0;
00181     for (const auto &item : ilist) {
00182         std::allocator_traits<Allocator>::construct(alloc, data + i++, item);
00183     }
00184     return *this;
00185 }
00186
00187 void assign(size_type count, const T &value) {
00188     clear();
00189     if (count > Acapacity) {
00190         if (data)
00191             alloc.deallocate(data, Acapacity);
00192         data = alloc.allocate(count);
00193         Acapacity = count;
00194     }
00195
00196     for (size_type i = 0; i < count; ++i) {
00197         std::allocator_traits<Allocator>::construct(alloc, data + i, value);
00198     }
00199     Asize = count;
00200 }
00201
00202 template <class InputIt, typename = typename std::enable_if<
00203     !std::is_integral<InputIt>::value>::type>
00204 void assign(InputIt first, InputIt last) {
00205     size_type count = std::distance(first, last);
00206
00207     T *newData = alloc.allocate(count);
00208
00209     for (size_type i = 0; i < count; ++i) {
00210         std::allocator_traits<Allocator>::construct(alloc, newData + i, *first);
00211         ++first;
00212     }
00213
00214     clear();
00215     if (data)
00216         alloc.deallocate(data, Acapacity);
00217
00218     data = newData;
00219     Asize = count;
00220     Acapacity = count;
00221 }
00222
00223 void assign(std::initializer_list<T> ilist) {
00224     size_type count = ilist.size();
00225     T *newData = alloc.allocate(count);
00226
00227     size_type i = 0;
00228     for (const auto &item : ilist) {
00229         std::allocator_traits<Allocator>::construct(alloc, newData + i++, item);
00230     }
00231
00232     clear();
00233     if (data)
00234         alloc.deallocate(data, Acapacity);
00235
00236     data = newData;
00237     Asize = count;
00238     Acapacity = count;
00239 }
00240
00241 template <class R> void assign_range(R &&rg) {
00242     assign(std::begin(rg), std::end(rg));
00243 }
00244
00245 void push_back(const T &value) {
00246     if (Asize >= Acapacity)
00247         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00248
00249     std::allocator_traits<Allocator>::construct(alloc, data + Asize, value);
00250     ++Asize;
00251 }
00252
00253 void push_back(T &&value) {
00254     if (Asize >= Acapacity)
00255         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00256

```

```

00257     std::allocator_traits<Allocator>::construct(alloc, data + Asize,
00258                                                std::move(value));
00259     ++Asize;
00260 }
00261
00262 reference operator[](size_type pos) { return data[pos]; }
00263
00264 const_reference operator[](size_type pos) const { return data[pos]; }
00265
00266 size_type size() const noexcept { return Asize; }
00267
00268 size_type capacity() const noexcept { return Acapacity; }
00269
00270 void reserve(size_type new_cap) {
00271     if (new_cap <= Acapacity)
00272         return;
00273
00274     T *newData = alloc.allocate(new_cap);
00275
00276     size_type i = 0;
00277
00278     try {
00279         for (; i < Asize; ++i) {
00280             std::allocator_traits<Allocator>::construct(
00281                 alloc, newData + i, std::move_if_noexcept(data[i]));
00282         }
00283     } catch (...) {
00284         for (size_type j = 0; j < i; ++j) {
00285             std::allocator_traits<Allocator>::destroy(alloc, newData + j);
00286         }
00287
00288         alloc.deallocate(newData, new_cap);
00289         throw;
00290     }
00291
00292     for (size_type i = 0; i < Asize; ++i) {
00293         std::allocator_traits<Allocator>::destroy(alloc, data + i);
00294     }
00295
00296     if (data) {
00297         alloc.deallocate(data, Acapacity);
00298     }
00299
00300     data = newData;
00301     Acapacity = new_cap;
00302 }
00303
00304 void pop_back() {
00305     if (Asize > 0) {
00306         --Asize;
00307         std::allocator_traits<Allocator>::destroy(alloc, data + Asize);
00308     }
00309 }
00310
00311 void clear() {
00312     for (size_type i = 0; i < Asize; ++i)
00313         std::allocator_traits<Allocator>::destroy(alloc, data + i);
00314
00315     Asize = 0;
00316 }
00317
00318 iterator begin() { return data; }
00319 const_iterator begin() const { return data; }
00320 const_iterator cbegin() const noexcept { return data; }
00321 iterator end() { return data + Asize; }
00322 const_iterator end() const { return data + Asize; }
00323 const_iterator cend() const noexcept { return data + Asize; }
00324 reverse_iterator rbegin() { return reverse_iterator(end()); }
00325 const_reverse_iterator rbegin() const {
00326     return const_reverse_iterator(end());
00327 }
00328 reverse_iterator rend() { return reverse_iterator(begin()); }
00329 const_reverse_iterator rend() const {
00330     return const_reverse_iterator(begin());
00331 }
00332 reverse_iterator crbegin() const noexcept {
00333     return const_reverse_iterator(end());
00334 }
00335 const_reverse_iterator crend() const noexcept {
00336     return const_reverse_iterator(begin());
00337 }
00338 reference front() {
00339     if (Asize == 0)
00340         throw std::out_of_range("Vector is empty");
00341     return data[0];
00342 }
00343 const_reference front() const {
00344     if (Asize == 0)

```

```

00344         throw std::out_of_range("Vector is empty");
00345     return data[0];
00346 }
00347 reference back() {
00348     if (Asize == 0)
00349         throw std::out_of_range("Vector is empty");
00350     return data[Asize - 1];
00351 }
00352 const_reference back() const {
00353     if (Asize == 0)
00354         throw std::out_of_range("Vector is empty");
00355     return data[Asize - 1];
00356 }
00357
00358 template <class... Args>
00359 iterator emplace(const_iterator pos, Args &&...args) {
00360     size_type index = pos - data;
00361     if (index > Asize)
00362         throw std::out_of_range("Range error");
00363
00364     if (Asize >= Acapacity)
00365         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00366
00367     for (size_type i = Asize; i > index; --i) {
00368         std::allocator_traits<Allocator>::construct(alloc, data + i,
00369                                                     std::move(data[i - 1]));
00370         std::allocator_traits<Allocator>::destroy(alloc, data + i - 1);
00371     }
00372
00373     std::allocator_traits<Allocator>::construct(alloc, data + index,
00374                                                 std::forward<Args>(args)...);
00375     ++Asize;
00376     return data + index;
00377 }
00378
00379 template <class... Args> reference emplace_back(Args &&...args) {
00380     if (Asize >= Acapacity)
00381         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00382
00383     std::allocator_traits<Allocator>::construct(alloc, data + Asize,
00384                                                 std::forward<Args>(args)...);
00385
00386     return data[Asize++];
00387 }
00388
00389 iterator insert(const_iterator pos, const T &value) {
00390     size_type index = pos - data;
00391
00392     if (index > Asize)
00393         throw std::out_of_range("Insert position out of range");
00394
00395     if (Asize >= Acapacity)
00396         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00397
00398     for (size_type i = Asize; i > index; --i) {
00399         std::allocator_traits<Allocator>::construct(
00400             alloc, data + i, std::move_if_noexcept(data[i - 1]));
00401
00402         std::allocator_traits<Allocator>::destroy(alloc, data + i - 1);
00403     }
00404
00405     std::allocator_traits<Allocator>::construct(alloc, data + index, value);
00406
00407     ++Asize;
00408     return data + index;
00409 }
00410 iterator insert(const_iterator pos, T &&value) {
00411     size_type index = pos - data;
00412
00413     if (index > Asize)
00414         throw std::out_of_range("Insert position out of range");
00415
00416     if (Asize >= Acapacity)
00417         reserve(Acapacity == 0 ? 1 : Acapacity * 2);
00418
00419     for (size_type i = Asize; i > index; --i) {
00420         std::allocator_traits<Allocator>::construct(
00421             alloc, data + i, std::move_if_noexcept(data[i - 1]));
00422         std::allocator_traits<Allocator>::destroy(alloc, data + i - 1);
00423     }
00424
00425     std::allocator_traits<Allocator>::construct(alloc, data + index,
00426                                                 std::move(value));
00427
00428     ++Asize;
00429     return data + index;
00430 }

```

```

00431
00432 iterator insert(const_iterator pos, size_type count, const T &value) {
00433     size_type index = pos - data;
00434
00435     if (index > Asize)
00436         throw std::out_of_range("Insert position out of range");
00437
00438     if (Asize + count > Acapacity)
00439         reserve((std::max)(Acapacity * 2, Asize + count));
00440
00441     for (size_type i = Asize + count; i > index + count; --i) {
00442         std::allocator_traits<Allocator>::construct(
00443             alloc, data + i - 1, std::move(data[i - count - 1]));
00444         std::allocator_traits<Allocator>::destroy(alloc, data + i - count - 1);
00445     }
00446
00447     for (size_type i = 0; i < count; ++i) {
00448         std::allocator_traits<Allocator>::construct(alloc, data + index + i,
00449                                                     value);
00450     }
00451
00452     Asize += count;
00453     return data + index;
00454 }
00455 template <class InputIt, typename = typename std::enable_if<
00456     !std::is_integral<InputIt>::value>::type>
00457 iterator insert(const_iterator pos, InputIt first, InputIt last) {
00458     size_type index = pos - data;
00459     size_type count = std::distance(first, last);
00460
00461     if (index > Asize)
00462         throw std::out_of_range("Insert position out of range");
00463
00464     if (Asize + count > Acapacity)
00465         reserve((std::max)(Acapacity * 2, Asize + count));
00466
00467     for (size_type i = Asize + count; i > index + count; --i) {
00468         std::allocator_traits<Allocator>::construct(
00469             alloc, data + i - 1, std::move(data[i - count - 1]));
00470         std::allocator_traits<Allocator>::destroy(alloc, data + i - count - 1);
00471     }
00472
00473     size_type i = 0;
00474     for (; first != last; ++first, ++i) {
00475         std::allocator_traits<Allocator>::construct(alloc, data + index + i,
00476                                                     *first);
00477     }
00478
00479     Asize += count;
00480     return data + index;
00481 }
00482 iterator insert(const_iterator pos, std::initializer_list<T> ilist) {
00483     return insert(pos, ilist.begin(), ilist.end());
00484 }
00485
00486 iterator erase(iterator pos) { return erase(const_iterator(pos)); }
00487 iterator erase(const_iterator pos) {
00488     size_type index = pos - data;
00489
00490     if (index >= Asize)
00491         throw std::out_of_range("Erase position out of range");
00492
00493     for (size_type i = index; i < Asize - 1; ++i) {
00494         data[i] = std::move(data[i + 1]);
00495     }
00496
00497     --Asize;
00498
00499     std::allocator_traits<Allocator>::destroy(alloc, data + Asize);
00500
00501     return data + index;
00502 }
00503 iterator erase(const_iterator first, const_iterator last) {
00504     size_type start = first - data;
00505     size_type end = last - data;
00506
00507     if (start > Asize || end > Asize || start > end)
00508         throw std::out_of_range("Erase range out of range");
00509
00510     size_type count = end - start;
00511
00512     for (size_type i = start; i < Asize - count; ++i) {
00513         data[i] = std::move(data[i + count]);
00514     }
00515
00516     for (size_type i = Asize - count; i < Asize; ++i) {
00517         std::allocator_traits<Allocator>::destroy(alloc, data + i);

```

```

00518     }
00519
00520     Asize -= count;
00521
00522     return data + start;
00523 }
00524 void shrink_to_fit() {
00525     if (Asize == 0) {
00526         clear();
00527
00528         if (data)
00529             alloc.deallocate(data, Acapacity);
00530
00531         data = nullptr;
00532         Acapacity = 0;
00533         return;
00534     }
00535
00536     if (Asize < Acapacity) {
00537         T *newData = alloc.allocate(Asize);
00538
00539         for (size_type i = 0; i < Asize; i++) {
00540             std::allocator_traits<Allocator>::construct(alloc, newData + i,
00541                                                         std::move(data[i]));
00542
00543             std::allocator_traits<Allocator>::destroy(alloc, data + i);
00544         }
00545
00546         alloc.deallocate(data, Acapacity);
00547
00548         data = newData;
00549         Acapacity = Asize;
00550     }
00551 }
00552 void resize(size_type count) {
00553     if (count < Asize) {
00554         for (size_type i = count; i < Asize; ++i)
00555             std::allocator_traits<Allocator>::destroy(alloc, data + i);
00556     } else if (count > Asize) {
00557         if (count > Acapacity) {
00558             reserve(count);
00559         }
00560         for (size_type i = Asize; i < count; ++i)
00561             std::allocator_traits<Allocator>::construct(alloc, data + i);
00562     }
00563     Asize = count;
00564 }
00565 void resize(size_type count, const T &value) {
00566     if (count < Asize) {
00567         for (size_type i = count; i < Asize; ++i)
00568             std::allocator_traits<Allocator>::destroy(alloc, data + i);
00569     } else if (count > Asize) {
00570         if (count > Acapacity) {
00571             reserve(count);
00572         }
00573         for (size_type i = Asize; i < count; ++i)
00574             std::allocator_traits<Allocator>::construct(alloc, data + i, value);
00575     }
00576     Asize = count;
00577 }
00578
00579 bool empty() const noexcept { return Asize == 0; }
00580
00581 T *data_ptr() { return data; }
00582
00583 const T *data_ptr() const { return data; }
00584
00585 reference at(size_type pos) {
00586     if (pos >= Asize)
00587         throw std::out_of_range("Vector index out of range");
00588     return data[pos];
00589 }
00590
00591 const_reference at(size_type pos) const {
00592     if (pos >= Asize)
00593         throw std::out_of_range("Vector index out of range");
00594     return data[pos];
00595 }
00596
00597 void swap(Vector &other) noexcept {
00598     std::allocator_traits<Allocator>::propagate_on_container_swap::value ||
00599     std::allocator_traits<Allocator>::is_always_equal::value) {
00600     if (this != &other) {
00601         std::swap(data, other.data);
00602         std::swap(Asize, other.Asize);
00603         std::swap(Acapacity, other.Acapacity);
00604     }

```

```

00605         if constexpr (std::allocator_traits<
00606                         Allocator>::propagate_on_container_swap::value) {
00607             std::swap(alloc, other.alloc);
00608         }
00609     }
00610 }
00611
00612 size_type max_size() const noexcept {
00613     return std::allocator_traits<Allocator>::max_size(alloc);
00614 }
00615
00616 friend bool operator==(const Vector &lhs, const Vector &rhs) {
00617     if (lhs.size() != rhs.size())
00618         return false;
00619
00620     for (size_type i = 0; i < lhs.size(); i++) {
00621         if (lhs[i] != rhs[i])
00622             return false;
00623     }
00624     return true;
00625 }
00626 friend bool operator!=(const Vector &lhs, const Vector &rhs) {
00627     return !(lhs == rhs);
00628 }
00629
00630 friend bool operator<(const Vector &lhs, const Vector &rhs) {
00631     return std::lexicographical_compare(lhs.begin(), lhs.end(), rhs.begin(),
00632                                         rhs.end());
00633 }
00634
00635 friend bool operator<=(const Vector &lhs, const Vector &rhs) {
00636     return !(rhs < lhs);
00637 }
00638
00639 friend bool operator>(const Vector &lhs, const Vector &rhs) {
00640     return rhs < lhs;
00641 }
00642
00643 friend bool operator>=(const Vector &lhs, const Vector &rhs) {
00644     return !(lhs < rhs);
00645 }
00646
00647 template <typename R> iterator insert_range(const_iterator pos, R &&rg) {
00648     return insert(pos, std::begin(rg), std::end(rg));
00649 }
00650
00651 template <typename R> void append_range(R &&rg) {
00652     insert_range(cend(), std::forward<R>(rg));
00653 }
00654
00655 friend void swap(Vector<T> &a, Vector<T> &b) noexcept { a.swap(b); }
00656
00657 Allocator get_allocator() const { return alloc; }
00658 };
00659
00660 #endif

```

7.9 include/Zmogus.h File Reference

```
#include <string>
```

Classes

- class [Zmogus](#)

7.10 Zmogus.h

[Go to the documentation of this file.](#)

```
00001 // Zmogus.h
```

```

00002 #pragma once
00003 #include <string>
00004
00008 class Zmogus {
00009 protected:
00010     std::string vardas_;
00011     std::string pavarde_;
00012
00013 public:
00014     Zmogus() : vardas_(""), pavarde_("") {}
00015
00016     Zmogus(const std::string &vardas, const std::string &pavarde)
00017         : vardas_(vardas), pavarde_(pavarde) {}
00018
00019     virtual ~Zmogus() {
00020         vardas_.clear();
00021         pavarde_.clear();
00022     }
00023
00024     virtual void WhoamI() const = 0;
00025
00026     std::string getVardas() const { return vardas_; }
00027     std::string getPavarde() const { return pavarde_; }
00028
00029     void setVardas(const std::string &v) { vardas_ = v; }
00030     void setPavarde(const std::string &p) { pavarde_ = p; }
00031 };

```

7.11 src/functions.cpp File Reference

Šiame faile yra įgyvendintos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

```

#include "functions.h"
#include "Studentas.h"
#include "timer.h"
#include <array>
#include <iostream>

```

Functions

- void [isvestis](#) (const [StudentuKonteineris](#) &studentai, bool arMediana)
- void [isvestisKonsole](#) (const [StudentuKonteineris](#) &studentai)
- void [isvestisFailas](#) (const [StudentuKonteineris](#) &studentai, string failoPavadinimas)
- string [skaitytiZodi](#) (const string &pranesimas)
- string [nuskaitytiVarda](#) ()
- string [nuskaitytiPavarde](#) ()
- [Studentas](#) [investisRanka](#) ()
- bool [suskaiciuotiGalutini](#) ([StudentuKonteineris](#) &studentai)
- void [suskaiciuotiStudentoGalutinius](#) ([Studentas](#) &studentas)
- void [suskaiciuotiGalutinius](#) ([StudentuKonteineris](#) &studentai)
- int [randomInt](#) (int min, int max)
- [Studentas](#) [generuotiStudenta](#) ()
- [Studentas](#) [generuotiPazymius](#) ()
- vector< int > [nuskaitytiPazymius](#) (istream &is, int namuDarbai)
- void [nuskaitytiFaila](#) ([StudentuKonteineris](#) &studentai, const string &failoPavadinimas)
- bool [rusiuotiPagalVarda](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [rusiuotiPagalPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [rusiuotiPagalVidurki](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [rusiuotiPagalMediana](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- void [rusiuotiStudentus](#) ([StudentuKonteineris](#) &studentai, int input)

- void [surusiuotiPagalPasirinkima](#) ([StudentuKonteineris](#) &studentai)
- int [skaitytiSkaiciu](#) (const string &pranesimas, int min, int max)
- string [generuotiFaila](#) (int studentuSkaicius, int namuDarbuSkaicius)
- void [skaidytiStudentus1](#) ([StudentuKonteineris](#) &studentai, [StudentuKonteineris](#) &vargsiukai, [StudentuKonteineris](#) &kietiakai)
- void [skaidytiStudentus2](#) ([StudentuKonteineris](#) &studentai, [StudentuKonteineris](#) &kietiakai)
- void [skaidytiStudentus3](#) ([StudentuKonteineris](#) &studentai, [StudentuKonteineris](#) &vargsiukai, [StudentuKonteineris](#) &kietiakai)
- void [failuGeneravimas](#) ()
- void [failoKurimoTestavimas](#) ()
- void [duomenuApdorojimoTestavimas](#) ()
- void [tikrinti](#) (bool condition)
- void [ruleOfFiveTestas](#) ()
- void [testuotiGreiti](#) ()
- void [failoNuskaitymas](#) ([StudentuKonteineris](#) &studentai)
- void [failoDuomenuApdorojimas](#) ([StudentuKonteineris](#) &studentai)
- void [studentuDuomenuApdorojimas](#) ([StudentuKonteineris](#) &studentai)

7.11.1 Detailed Description

Šiame faile yra įgyvendintos įvairios funkcijos, skirtos studentų duomenų apdorojimui, įskaitant duomenų įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

Šios funkcijos apima:

- Duomenų įvedimą ranka,
- Atsitiktinių pažymių generavimą,
- Studentų vardų, pavardžių ir pažymių generavimą,
- Failų nuskaitymą ir duomenų apdorojimą,
- Duomenų išvedimą į konsolę ir failus,
- Studentų rūšiavimą pagal įvairius kriterijus,
- Greičio testavimą naudojant [Timer](#) klasę.

Šios funkcijos yra pagrindinės programos dalys, leidžiančios vartotojui efektyviai dirbti su studentų duomenimis ir atlikti įvairias operacijas su jais.

7.11.2 Function Documentation

7.11.2.1 [duomenuApdorojimoTestavimas\(\)](#)

```
void duomenuApdorojimoTestavimas ()
```

7.11.2.2 [failoDuomenuApdorojimas\(\)](#)

```
void failoDuomenuApdorojimas (  
    StudentuKonteineris & studentai)
```

7.11.2.3 failoKurimoTestavimas()

```
void failoKurimoTestavimas ()
```

7.11.2.4 failoNuskaitymas()

```
void failoNuskaitymas (  
    StudentuKonteineris & studentai)
```

7.11.2.5 failuGeneravimas()

```
void failuGeneravimas ()
```

7.11.2.6 generuotiFaila()

```
string generuotiFaila (  
    int studentuSkaicius,  
    int namuDarbuSkaicius)
```

7.11.2.7 generuotiPazymius()

```
Studentas generuotiPazymius ()
```

7.11.2.8 generuotiStudenta()

```
Studentas generuotiStudenta ()
```

7.11.2.9 isvestis()

```
void isvestis (  
    const StudentuKonteineris & studentai,  
    bool arMediana)
```

7.11.2.10 isvestisFailas()

```
void isvestisFailas (  
    const StudentuKonteineris & studentai,  
    string failoPavadinimas)
```

7.11.2.11 isvestisKonsole()

```
void isvestisKonsole (  
    const StudentuKonteineris & studentai)
```

7.11.2.12 ivestisRanka()

```
Studentas ivestisRanka ()
```

7.11.2.13 nuskaitytiFaila()

```
void nuskaitytiFaila (  
    StudentuKonteineris & studentai,  
    const string & failoPavadinimas)
```

7.11.2.14 nuskaitytiPavarde()

```
string nuskaitytiPavarde ()
```

7.11.2.15 nuskaitytiPazymius()

```
vector< int > nuskaitytiPazymius (  
    istream & is,  
    int namuDarbai)
```

7.11.2.16 nuskaitytiVarda()

```
string nuskaitytiVarda ()
```

7.11.2.17 randomInt()

```
int randomInt (  
    int min,  
    int max)
```

7.11.2.18 ruleOfFiveTestas()

```
void ruleOfFiveTestas ()
```

7.11.2.19 rusiuotiPagalMediana()

```
bool rusiuotiPagalMediana (  
    const Studentas & a,  
    const Studentas & b)
```

7.11.2.20 rusiuotiPagalPavarde()

```
bool rusiuotiPagalPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

7.11.2.21 rusiuotiPagalVarda()

```
bool rusiuotiPagalVarda (  
    const Studentas & a,  
    const Studentas & b)
```

7.11.2.22 rusiuotiPagalVidurki()

```
bool rusiuotiPagalVidurki (  
    const Studentas & a,  
    const Studentas & b)
```

7.11.2.23 rusiuotiStudentus()

```
void rusiuotiStudentus (  
    StudentuKonteineris & studentai,  
    int input)
```

7.11.2.24 skaidytiStudentus1()

```
void skaidytiStudentus1 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & vargsiukai,  
    StudentuKonteineris & kietiakai)
```

7.11.2.25 skaidytiStudentus2()

```
void skaidytiStudentus2 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & kietiakai)
```

7.11.2.26 skaidytiStudentus3()

```
void skaidytiStudentus3 (  
    StudentuKonteineris & studentai,  
    StudentuKonteineris & vargsiukai,  
    StudentuKonteineris & kietiakai)
```

7.11.2.27 skaitytiSkaiciu()

```
int skaitytiSkaiciu (  
    const string & pranesimas,  
    int min,  
    int max)
```

7.11.2.28 skaitytiZodi()

```
string skaitytiZodi (  
    const string & pranesimas)
```

7.11.2.29 studentuDuomenuApdorojimas()

```
void studentuDuomenuApdorojimas (  
    StudentuKonteineris & studentai)
```

7.11.2.30 surusiuotiPagalPasirinkima()

```
void surusiuotiPagalPasirinkima (  
    StudentuKonteineris & studentai)
```

7.11.2.31 suskaiciuotiGalutini()

```
bool suskaiciuotiGalutini (  
    StudentuKonteineris & studentai)
```

7.11.2.32 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (  
    StudentuKonteineris & studentai)
```

7.11.2.33 suskaiciuotiStudentoGalutinius()

```
void suskaiciuotiStudentoGalutinius (  
    Studentas & studentas)
```

7.11.2.34 testuotiGreiti()

```
void testuotiGreiti ()
```

7.11.2.35 tikrinti()

```
void tikrinti (  
    bool condition)
```

7.12 src/main.cpp File Reference

```
#include "Studentas.h"  
#include "functions.h"  
#include "timer.h"
```

Functions

- `int main ()`

Pagrindinė programos funkcija, kuri leidžia vartotojui pasirinkti įvairias operacijas su studentų duomenimis, įskaitant įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

7.12.1 Function Documentation

7.12.1.1 main()

```
int main ()
```

Pagrindinė programos funkcija, kuri leidžia vartotojui pasirinkti įvairias operacijas su studentų duomenimis, įskaitant įvedimą, generavimą, failų skaitymą ir rašymą, bei greičio testavimą.

Vartotojas gali pasirinkti: 1 - įvesti studento duomenis ranka, 2 - generuoti tik pažymius, 3 - generuoti studento vardus, pavardes ir pažymius, 4 - nuskaityti studentų duomenis iš failo, 5 - generuoti failą su studentų duomenimis, 6 - testuoti programos greitį, 7 - baigti darbą.

Programa naudoja [Studentas](#) klasę, kuri paveldi iš abstrakčios [Zmogus](#) klasės, ir įvairias funkcijas duomenų apdorojimui bei failų operacijoms.

Returns

`int` Programos pabaigos kodas (0 reiškia sėkmingą pabaigą).

7.13 src/Studentas.cpp File Reference

```
#include "Studentas.h"
#include <algorithm>
#include <iomanip>
#include <numeric>
#include <ostream>
#include <sstream>
#include <string>
```

Functions

- `ostream & operator<< (ostream &out, const Studentas &studentas)`

Šiame faile yra įgyvendinami [Studentas](#) klasės metodai, įskaitant konstruktorius, priskyrimo operatorius, destruktorius, bei perdengtus įvedimo ir išvedimo operatorius. Taip pat yra metodai pažymių tvarkymui ir galutinių rezultatų skaičiavimui.

- `istream & operator>> (std::istream &in, Studentas &s)`

7.13.1 Function Documentation

7.13.1.1 operator<<()

```
ostream & operator<< (  
    ostream & out,  
    const Studentas & studentas)
```

Šiame faile yra įgyvendinami `Studentas` klasės metodai, įskaitant konstruktorius, priskyrimo operatorius, destruktorius, bei perdengtus įvedimo ir išvedimo operatorius. Taip pat yra metodai pažymių tvarkymui ir galutinių rezultatų skaičiavimui.

Ši klasė paveldi iš abstrakčios `Zmogus` klasės, todėl turi įgyvendinti `WhoamI` metodą. Klasė taip pat turi savo specifinius duomenis, tokius kaip namų darbų skaičius, pažymiai, egzamino balas, galutinis vidurkis ir mediana.

Konstruktoriai ir priskyrimo operatoriai užtikrina tinkamą resursų valdymą, o destruktorius užtikrina, kad visi resursai būtų tinkamai atlaisvinti.

Perdengti įvedimo ir išvedimo operatoriai leidžia lengvai skaityti ir rašyti `Studentas` objektus naudojant standartinius srautus.

Ši klasė yra testuojama naudojant Google Test framework'ą, siekiant užtikrinti, kad visi metodai veiktų teisingai įvairiais scenarijais.

7.13.1.2 operator>>()

```
istream & operator>> (  
    std::istream & in,  
    Studentas & s)
```


Index

- ~Studentas
 - Studentas, [12](#)
- ~Vector
 - Vector< T >, [21](#)
- ~Zmogus
 - Zmogus, [30](#)
- addPazymys
 - Studentas, [13](#)
- Allocator
 - Vector< T >, [18](#)
- append_range
 - Vector< T >, [21](#)
- assign
 - Vector< T >, [21](#)
- assign_range
 - Vector< T >, [21](#)
- at
 - Vector< T >, [22](#)
- back
 - Vector< T >, [22](#)
- begin
 - Vector< T >, [22](#)
- capacity
 - Vector< T >, [22](#)
- cbegin
 - Vector< T >, [22](#)
- cend
 - Vector< T >, [23](#)
- clear
 - Vector< T >, [23](#)
- Comparator
 - Studentas.h, [40](#)
- const_iterator
 - Vector< T >, [18](#)
- const_reference
 - Vector< T >, [19](#)
- const_reverse_iterator
 - Vector< T >, [19](#)
- crbegin
 - Vector< T >, [23](#)
- crend
 - Vector< T >, [23](#)
- data_ptr
 - Vector< T >, [23](#)
- duomenuApdorojimoTestavimas
 - functions.cpp, [53](#)
- functions.h, [34](#)
- elapsed
 - Timer, [16](#)
- emplace
 - Vector< T >, [23](#)
- emplace_back
 - Vector< T >, [23](#)
- empty
 - Vector< T >, [24](#)
- end
 - Vector< T >, [24](#)
- erase
 - Vector< T >, [24](#)
- failoDuomenuApdorojimas
 - functions.cpp, [53](#)
 - functions.h, [34](#)
- failoKurimoTestavimas
 - functions.cpp, [53](#)
 - functions.h, [35](#)
- failoNuskaitymas
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- failuGeneravimas
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- front
 - Vector< T >, [24](#)
- functions.cpp
 - duomenuApdorojimoTestavimas, [53](#)
 - failoDuomenuApdorojimas, [53](#)
 - failoKurimoTestavimas, [53](#)
 - failoNuskaitymas, [54](#)
 - failuGeneravimas, [54](#)
 - generuotiFaila, [54](#)
 - generuotiPazymius, [54](#)
 - generuotiStudenta, [54](#)
 - isvestis, [54](#)
 - isvestisFailas, [54](#)
 - isvestisKonsole, [54](#)
 - investisRanka, [54](#)
 - nuskaitytiFaila, [55](#)
 - nuskaitytiPavarde, [55](#)
 - nuskaitytiPazymius, [55](#)
 - nuskaitytiVarda, [55](#)
 - randomInt, [55](#)
 - ruleOfFiveTestas, [55](#)
 - rusiuotiPagalMediana, [55](#)
 - rusiuotiPagalPavarde, [55](#)

- rusiuotiPagalVarda, [55](#)
- rusiuotiPagalVidurki, [56](#)
- rusiuotiStudentus, [56](#)
- skaitytiStudentus1, [56](#)
- skaitytiStudentus2, [56](#)
- skaitytiStudentus3, [56](#)
- skaitytiSkaiciu, [56](#)
- skaitytiZodi, [56](#)
- studentuDuomenuApdorojimas, [57](#)
- surusiuotiPagalPasirinkima, [57](#)
- suskaiciuotiGalutini, [57](#)
- suskaiciuotiGalutinius, [57](#)
- suskaiciuotiStudentoGalutinius, [57](#)
- testuotiGreiti, [57](#)
- tikrinti, [57](#)
- functions.h
 - duomenuApdorojimoTestavimas, [34](#)
 - failoDuomenuApdorojimas, [34](#)
 - failoKurimoTestavimas, [35](#)
 - failoNuskaitymas, [35](#)
 - failuGeneravimas, [35](#)
 - generuotiFaila, [35](#)
 - generuotiPazymius, [35](#)
 - generuotiStudenta, [35](#)
 - isvestis, [35](#)
 - isvestisFailas, [35](#)
 - isvestisKonsole, [35](#)
 - investisRanka, [36](#)
 - nuskaitytiFaila, [36](#)
 - nuskaitytiPavarde, [36](#)
 - nuskaitytiPazymius, [36](#)
 - nuskaitytiVarda, [36](#)
 - ruleOfFiveTestas, [36](#)
 - rusiuotiPagalMediana, [36](#)
 - rusiuotiPagalPavarde, [36](#)
 - rusiuotiPagalVarda, [36](#)
 - rusiuotiPagalVidurki, [37](#)
 - rusiuotiStudentus, [37](#)
 - skaitytiStudentus1, [37](#)
 - skaitytiStudentus2, [37](#)
 - skaitytiStudentus3, [37](#)
 - skaitytiSkaiciu, [37](#)
 - skaitytiZodi, [37](#)
 - studentuDuomenuApdorojimas, [38](#)
 - surusiuotiPagalPasirinkima, [38](#)
 - suskaiciuotiGalutini, [38](#)
 - suskaiciuotiGalutinius, [38](#)
 - suskaiciuotiStudentoGalutinius, [38](#)
 - testuotiGreiti, [38](#)
 - tikrinti, [38](#)
- gautiVidurkiMediana
 - Studentas, [13](#)
- gautiVidurkiVidutini
 - Studentas, [13](#)
- generuotiFaila
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- generuotiPazymius
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- functions.cpp, [54](#)
- functions.h, [35](#)
- generuotiStudenta
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- get_allocator
 - Vector< T >, [25](#)
- getEgzaminoBalas
 - Studentas, [13](#)
- getGalutinisMediana
 - Studentas, [13](#)
- getGalutinisVidurkis
 - Studentas, [13](#)
- getNamuDarbai
 - Studentas, [13](#)
- getPavarde
 - Studentas, [13](#)
 - Zmogus, [30](#)
- getPazymiai
 - Studentas, [13](#)
- getVardas
 - Studentas, [14](#)
 - Zmogus, [30](#)
- include Directory Reference, [9](#)
- include/functions.h, [33](#), [39](#)
- include/Studentas.h, [39](#), [41](#)
- include/timer.h, [43](#)
- include/Vector.h, [43](#), [44](#)
- include/Zmogus.h, [51](#)
- insert
 - Vector< T >, [25](#)
- insert_range
 - Vector< T >, [25](#)
- isvestis
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- isvestisFailas
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- isvestisKonsole
 - functions.cpp, [54](#)
 - functions.h, [35](#)
- iterator
 - Vector< T >, [19](#)
- investisRanka
 - functions.cpp, [54](#)
 - functions.h, [36](#)
- main
 - main.cpp, [58](#)
- main.cpp
 - main, [58](#)
- max_size
 - Vector< T >, [26](#)
- nuskaitytiFaila
 - functions.cpp, [55](#)
 - functions.h, [36](#)

nuskaitytiPavarde
 functions.cpp, 55
 functions.h, 36
nuskaitytiPazymius
 functions.cpp, 55
 functions.h, 36
nuskaitytiVarda
 functions.cpp, 55
 functions.h, 36

operator!=
 Vector< T >, 28
operator<
 Vector< T >, 28
operator<<
 Studentas, 15
 Studentas.cpp, 59
operator<=
 Vector< T >, 28
operator>
 Vector< T >, 29
operator>>
 Studentas, 15
 Studentas.cpp, 59
operator>=
 Vector< T >, 29
operator=
 Studentas, 14
 Vector< T >, 26
operator==
 Vector< T >, 28
operator[]
 Vector< T >, 26

pavarde_
 Zmogus, 31
pop_back
 Vector< T >, 26
push_back
 Vector< T >, 26, 27

randomInt
 functions.cpp, 55
rbegin
 Vector< T >, 27
reference
 Vector< T >, 19
rend
 Vector< T >, 27
reserve
 Vector< T >, 27
reset
 Timer, 16
resize
 Vector< T >, 27
reverse_iterator
 Vector< T >, 19
ruleOfFiveTestas
 functions.cpp, 55
 functions.h, 36
rusiuotiPagalMediana
 functions.cpp, 55
 functions.h, 36
rusiuotiPagalPavarde
 functions.cpp, 55
 functions.h, 36
rusiuotiPagalVarda
 functions.cpp, 55
 functions.h, 36
rusiuotiPagalVidurki
 functions.cpp, 56
 functions.h, 37
rusiuotiStudentus
 functions.cpp, 56
 functions.h, 37

setEgzaminoBalas
 Studentas, 14
setGalutinisMediana
 Studentas, 14
setGalutinisVidurkis
 Studentas, 14
setNamuDarbai
 Studentas, 14
setPavarde
 Studentas, 14
 Zmogus, 30
setPazymiai
 Studentas, 14
setVardas
 Studentas, 15
 Zmogus, 31
shrink_to_fit
 Vector< T >, 28
size
 Vector< T >, 28
size_type
 Vector< T >, 19
skaidytiStudentus1
 functions.cpp, 56
 functions.h, 37
skaidytiStudentus2
 functions.cpp, 56
 functions.h, 37
skaidytiStudentus3
 functions.cpp, 56
 functions.h, 37
skaitytiSkaiciu
 functions.cpp, 56
 functions.h, 37
skaitytiZodi
 functions.cpp, 56
 functions.h, 37
src Directory Reference, 9
src/functions.cpp, 52
src/main.cpp, 57
src/Studentas.cpp, 58
Studentas, 11

- ~Studentas, 12
- addPazymys, 13
- gautiVidurkiMediana, 13
- gautiVidurkiVidutini, 13
- getEgzaminoBalas, 13
- getGalutinisMediana, 13
- getGalutinisVidurkis, 13
- getNamuDarbai, 13
- getPavarde, 13
- getPazymiai, 13
- getVardas, 14
- operator<<, 15
- operator>>, 15
- operator=, 14
- setEgzaminoBalas, 14
- setGalutinisMediana, 14
- setGalutinisVidurkis, 14
- setNamuDarbai, 14
- setPavarde, 14
- setPazymiai, 14
- setVardas, 15
- Studentas, 12
- surusiوتيPazymius, 15
- WhoamI, 15
- Studentas.cpp
 - operator<<, 59
 - operator>>, 59
- Studentas.h
 - Comparator, 40
 - StudentuKonteineris, 40
 - surusiوتيStudentus, 40
- studentuDuomenuApdorojimas
 - functions.cpp, 57
 - functions.h, 38
- StudentuKonteineris
 - Studentas.h, 40
- surusiوتيPagalPasirinkima
 - functions.cpp, 57
 - functions.h, 38
- surusiوتيPazymius
 - Studentas, 15
- surusiوتيStudentus
 - Studentas.h, 40
- suskaiciوتيGalutini
 - functions.cpp, 57
 - functions.h, 38
- suskaiciوتيGalutinius
 - functions.cpp, 57
 - functions.h, 38
- suskaiciوتيStudentoGalutinius
 - functions.cpp, 57
 - functions.h, 38
- swap
 - Vector< T >, 28, 29
- testuotiGreiti
 - functions.cpp, 57
 - functions.h, 38
- tikrinti
 - functions.cpp, 57
 - functions.h, 38
- Timer, 15
 - elapsed, 16
 - reset, 16
 - Timer, 16
- type
 - Vector< T >, 19
- vardas_
 - Zmogus, 31
- Vector
 - Vector< T >, 20, 21
- Vector< T >, 16
 - ~Vector, 21
 - Allocator, 18
 - append_range, 21
 - assign, 21
 - assign_range, 21
 - at, 22
 - back, 22
 - begin, 22
 - capacity, 22
 - cbegin, 22
 - cend, 23
 - clear, 23
 - const_iterator, 18
 - const_reference, 19
 - const_reverse_iterator, 19
 - crbegin, 23
 - crend, 23
 - data_ptr, 23
 - emplace, 23
 - emplace_back, 23
 - empty, 24
 - end, 24
 - erase, 24
 - front, 24
 - get_allocator, 25
 - insert, 25
 - insert_range, 25
 - iterator, 19
 - max_size, 26
 - operator!=, 28
 - operator<, 28
 - operator<=, 28
 - operator>, 29
 - operator>=, 29
 - operator=, 26
 - operator==, 28
 - operator[], 26
 - pop_back, 26
 - push_back, 26, 27
 - rbegin, 27
 - reference, 19
 - rend, 27
 - reserve, 27
 - resize, 27
 - reverse_iterator, 19

shrink_to_fit, [28](#)
size, [28](#)
size_type, [19](#)
swap, [28](#), [29](#)
type, [19](#)
Vector, [20](#), [21](#)

Whoaml

Studentas, [15](#)
Zmogus, [31](#)

Zmogus, [29](#)

~Zmogus, [30](#)
getPavarde, [30](#)
getVardas, [30](#)
pavarde_, [31](#)
setPavarde, [30](#)
setVardas, [31](#)
vardas_, [31](#)
Whoaml, [31](#)
Zmogus, [30](#)